SETI B4

Embedded Linux

Valerii DROBOTUN
Yasser BOURSSIA
Sirine MAMA

Télécom Paris
Institut Polytechnique de Paris & Université Paris-Saclay

January 2023

Contents

Introduction **2**

1 TP1: Minimal Linux/initramfs image and bootloader **3**
1.1 Intro . . . 3
1.2 QEMU and cross-compilation . . . 3
1.3 Linux kernel compilation . . . 3
1.4 Linux kernel launch . . . 4
1.5 Busybox . . . 7
1.6 U-Boot bootloader . . . 8
1.7 Complete minimal system . . . 9
1.8 Conclusion . . . 11

2 TP2: First driver **12**
2.1 Intro . . . 12
2.2 Prerequisites . . . 12
2.3 Driver skeleton . . . 12
2.4 Making the driver more useful . . . 14
2.5 ADXL345 first I^2C interaction . . . 16
2.6 ADXL345 setup . . . 16
2.7 Conclusion . . . 16

3 TP3: Driver access from user space **21**
3.1 Intro . . . 21
3.2 Misc framework description . . . 21
3.3 Misc framework in our driver . . . 21
3.4 Test program . . . 22
3.5 Calls to `ioctl` . . . 24
3.6 File isolation for different processes . . . 25
3.7 Conclusion . . . 26

4 TP4: Interrupts **31**
4.1 Intro . . . 31
4.2 Update of the ADXL345 setup . . . 31
4.3 Interrupt handling . . . 32
4.4 Test program . . . 34
4.5 Conclusion . . . 36

References **38**

Introduction

Embedded Linux is a type of Linux operating system/kernel that was designed to be installed and used in embedded devices or systems.
Embedded Linux is adaptable, affordable, open-source, and works with special-purpose microprocessors. With a stable kernel and the flexibility to read, change, and redistribute source code, Linux has several software, development, and support providers available in comparison to proprietary embedded operating systems. Additionally, it enables a very modular building block approach to creating a unique system, increasing versatility. It has become very popular among the embedded systems engineers.

The aim of our project is:

- Compile and load our first kernel module.
- Write a driver for an I2C device as well as initialize the device when detected and reset the device when it is removed.
- Add the possibility for the user space to interact with the ADXL345 device driver via the misc framework.

1 TP1: Minimal Linux/initramfs image and bootloader

1.1 Intro

During this practical work, we are learning how to create a minimal system to run Linux. It includes the kernel compilation, then adding some flavor of kernel environment, namely **BusyBox**, default `initramfs` file system, and finally secondary bootloader, called **U-Boot** or **Das Boot** if you wish.

1.2 QEMU and cross-compilation

To begin with, we are not going to run the compiled kernel/environment modules/etc on the real physical system. Instead, we are using a simulator called **QEMU**. It allows running non-native applications on any supported platform, `x86-64` in our case. More specifically, we have a slightly modified QEMU build: it simulates not only the specific ARM processor but the whole ARM board of type "Arm Versatile Express board", also known as `vexpress-a9`.

The only thing we should keep in mind – we cannot use the default `gcc` compiler that goes bundled with our host Linux system. Apparently, `x86` assembler does not work on ARM processors. More precisely, we have to follow the cross-compilation (even if my PC will be on ARM one day, it would still require cross-compilation, because it would be ARM64 for sure). That's why we have to install `gcc-linaro` development pack with `arm-linux-gnueabihf-gcc` compilator, which outputs binaries dedicated to the **Cortex-A9** processors. We are going to use it to compile the Linux kernel, Busybox environment, and U-boot bootloader during the next steps.

1.3 Linux kernel compilation

First things first, we are compiling our Linux kernel. The process is quite straightforward, requiring only installing some Linux packages to compile the kernel. More precisely, we should have at least `build-essential, libncurses5-dev, file, cpio, unzip, rsync, bc` installed on the host system.

Do not forget to set the `PATH`, `ARCH` and `CROSS_COMPILE` variables:

```
export PATH=../gcc-linaro-7.4.1-2019.02-x86_64_arm-linux-gnueabihf/bin/:$PATH
export ARCH=arm
export CROSS_COMPILE=arm-linux-gnueabihf-
```

Remember, we are trying to compile binaries for the platform, different from the host one, so we have to use our specific compiler, to let our host system know where to find it. In addition, we are setting the `ARCH` variable, because we intend to build the kernel for the ARM system.

Then, we are cloning the official Linux repository and choosing a bit older kernel, version 5.x.x to ensure compatibility. More modern version 6.x.x seems to be a bit unstable for now. After downloading the kernel, we just need to update the build options by executing `make vexpress_defconfig` to set default parameters, specific for our devboard, and then `make menuconfig`. Actually, all the defaults are fine, we only have to update the only one option, called **udev enable**, shown in the figure 1. Basically, uevent or *User Events* are a way to interact between kernel or especially drivers, running in the kernel space, and user space.

Finally, kernel build is made with `make install`, probably with `-j` option to enable multi-core compilation. Luckily, the target board doesn't have too many devices, which means that not so modules should be compiled. That results in an appropriate compile time, around 10 minutes on the notebook processor, compared to 3+ hours for the x86 system kernel.

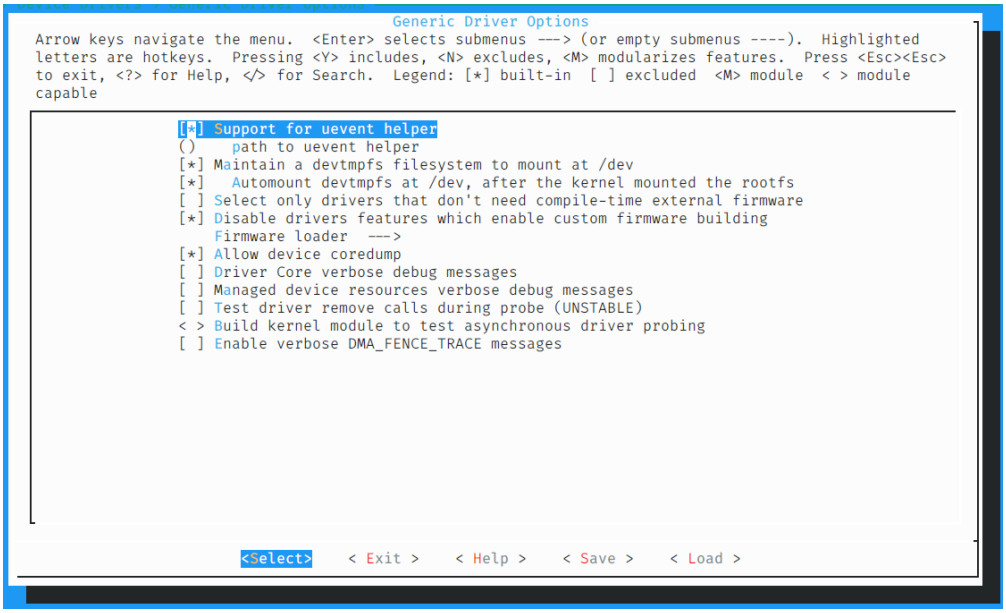


Figure 1: Pseudographical kernel compilation setup

Some errors could appear after launching the Makefile, telling that there are no header files found and compilation cannot proceed. Usually, that means some additional Linux packages should be installed. Any kind of such problem could be solved by copying `make` output to the StackOverflow search and finding the first response with the list of packages required to install. That simply works.

In my proper case, I had everything necessary already installed, but `make` was not happy because there were MPEG libraries missing.

As a result of the compilation, we obtain two necessary files:

- `zImage` – compressed kernel image (**z** stands for **zip**)
- `vexpress-v2p-ca9.dtb` – compiled device tree for our board.

We are going to launch the kernel in the next section.

1.4 Linux kernel launch

To launch our simulation, we only have to define the machine type for QEMU, and then pass two files, generated during the previous step. It can be done as follows:

```
$ qemu-system-arm -machine vexpress-a9 -nographic \
-kernel linux-5.15.6/build/arch/arm/boot/zImage \
-dtb linux-5.15.6/build/arch/arm/boot/dts/vexpress-v2p-ca9.dtb
```

Note that we are disabling the graphical mode for two reasons: we want to run the simplest system, so we need only the terminal access, and more important – we don't have a graphical shell, only a bare Linux kernel, even without environment.

So simulation started, from the log it could be observed that the system was booting and the kernel was loaded and started.

The startup procedure mainly performs the following steps:

- Detect the CPU kernel is running on
- Initialize memory and DMA
- Initialize logging buffer
- Initialize memory allocator (SLUB)
- RCU initialization (synchronization mechanisms)
- Initialization of interrupt handlers
- Memory cache configuration
- Random generator configuration
- Clock generator initialization
- Spectre/Meltdown patch apply
- Multicore activation
- SCSI initialization
- USB driver init
- TCP/IP stack initialization
- Audio stack initialization (AC'97/ALSA)

But in the end, the kernel panicked, as can be seen in figure 2. This happens because it cannot mount rootfs, which have nowhere to mount actually.

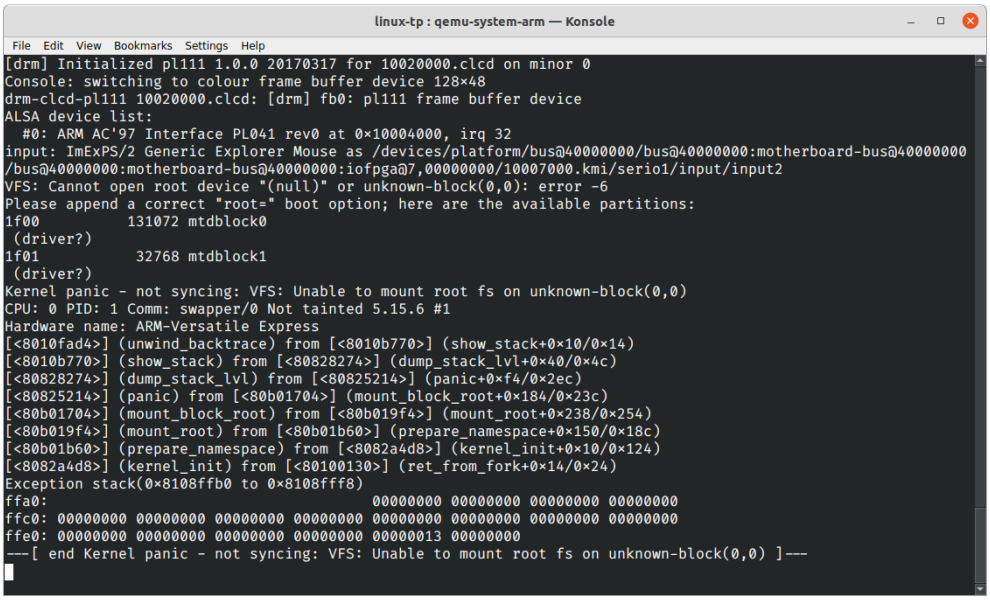


Figure 2: Kernel panic message

```c
#include <stdio.h>
#include <unistd.h>

int main (int argc, char** argv) {
    printf("Hello from init\n");
    sleep(10);
    return 0;
}
```

Listing 1: Simple `init`

So the next step is to provide a filesystem to the kernel, with the existing `init` program. In any other case, the kernel will panic – it always looks for `init`, because it is the first process to be launched, as an ancestor of all the following processes. We then have to write the init code, presented in the listing 1. This code basically does nothing, except print to the standard output, wait for 10 seconds, and terminate. This will allow us to see the message before the kernel panic.

It is left only to compile `init` with our cross compiler with the following command:

```
$ arm-linux-gnueabihf-gcc -static init.c -o init.
```

We should use `static` option because otherwise, it won't be possible to launch the program: it will require shared libraries, that should be loaded by *someone else*, who doesn't exist yet in our build. With such an option, we ensure that the code of libraries will be integrated into the same binary.

Then, the output binary file should be zipped in the following way:

```
$ echo init | cpio -o -H newc | gzip > test.cpio.gz
```

The `cpio` program was used to create filesystems on the tape, but now it's used to create filesystems for the kernel. The `-o` option tells it to read standard input and create an archive on standard output. Option `H` stands for the format definition, in our case **newc** says that we are using SVR4 format, which supports more than 65536 i-nodes. Finally, the archive is passed to `gzip` to create a compressed one.

With newly created filesystem, we can launch our simulation again to ensure that kernel launches the `init` process:

```
$ qemu-system-arm -machine vexpress-a9 -nographic \
interrupt -kernel linux-5.15.6/build/arch/arm/boot/zImage \
-dtb linux-5.15.6/build/arch/arm/boot/dts/vexpress-v2p-ca9.dtb \
-initrd intramfs/test.cpio.gz.
```

Note that this time we passed our archive with `init` in it via `-initrd` option, denoting the rootfs. After launch, we see the same initialization procedure, but with the print of `Hello from init`, shown in figure 3. At this moment we can be sure that `init` was launched. For sure, after 10 seconds after the print, we observe another kernel panic message, telling that the system attempted to kill `init`, which is normally not expected. Since the `init` is an ancestor of other processes, it should never exit, as it was made in our case. Anyway, it is sufficient to know that our system starts the kernel and launches something.

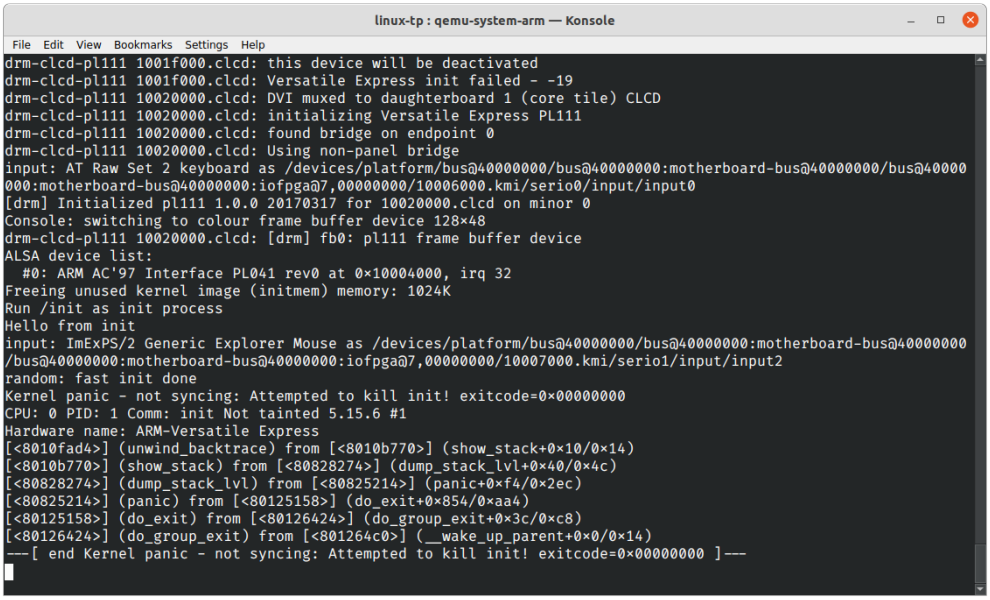


Figure 3: Hello init and kernel panic

1.5 Busybox

In this part, we will be using **BusyBox**, because in fact, our Linux kernel cannot do anything on its own – it doesn't even work with the terminal. It needs environmental utilities to do so (and a lot more).

BusyBox includes simplified (very reduced functionality) versions of many Unix utilities such as the `init`, `sh`, `mount`, `cp`, `ls` and so on into one small executable, which can be very practical for small embedded systems.

We have to first prepare the tree structure (the directories for the **BusyBox** image), as presented in the figure 4.

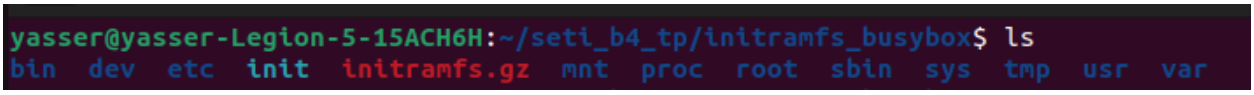


Figure 4: BusyBox Tree Structure

We then configure some files; by configuring the **rcS** file, which does the following:

- Mounts the file system, described in the `/etc/fstab` file.
- Activates `mdev`.

After the creation of the aforementioned file (and the `fstab/inittab` files), we can then create the full image using the `cpio` command:

```
find .  | cpio -o -H newc -R +0:+0 | gzip > initramfs.gz
```

This command is used the same as it is used before, that is, creating an archive, in this case, we are giving it the output of the `find` command, which actually outputs all files in the current

directory. We add an external argument `-R`, which simply gives the ownership of the archive to the **0** user. Then passing the result to `gzip` to compress our archive.

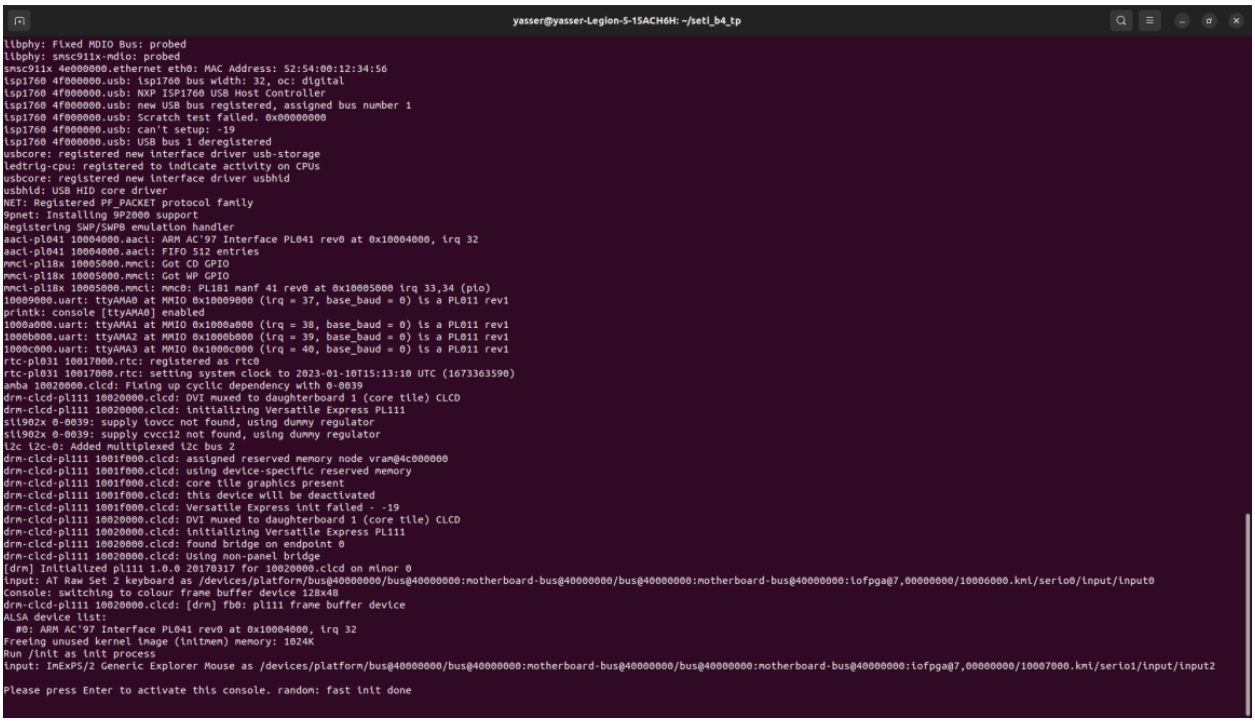


Figure 5: Execution of BusyBox image on QEMU.

As seen in figure 5, the kernel image has been correctly executed on the ARM core. With no kernel panics in sight, we have actually provided it with the necessary components for a successful launch (kernel, the image with correct `init` and the utilities provided by **BusyBox**).

1.6 U-Boot bootloader

Before, we used the QEMU's built-in bootloader, which is not that close to the real bare system. To be able to start our kernel like on the real system, we have to use any kind of bootloader.

The most known one for embedded Linux is U-Boot, used by almost every project, even with Yocto/Buildroot. It is very flexible in terms of selecting of booting device and supports almost every platform you can imagine. Moreover, it is distributed under the GNU GPL license, with free access to the source code and the ability to modify the code if needed.

After downloading and extracting the **U-boot** bootloader, it should be compiled again for our target platform:

```
$ export ARCH=arm
$ export CROSS_COMPILE=arm-linux-gnueabihf-
$ make vexpress_ca9x4_defconfig
$ make.
```

The resulting binary file could be finally launched by QEMU as follows:

```
./qemu-system-arm -machine vexpress-a9 -nographic -kernel u-boot-2020.10/u-boot
```

As it can be seen in figure 6, showing the result of the execution, bootloader run ends with an error. This is pretty expected, as we haven't provided the bootloader with a kernel image to load. Actually it tries all available physical partitions, and then all available network cards to retrieve the kernel image.

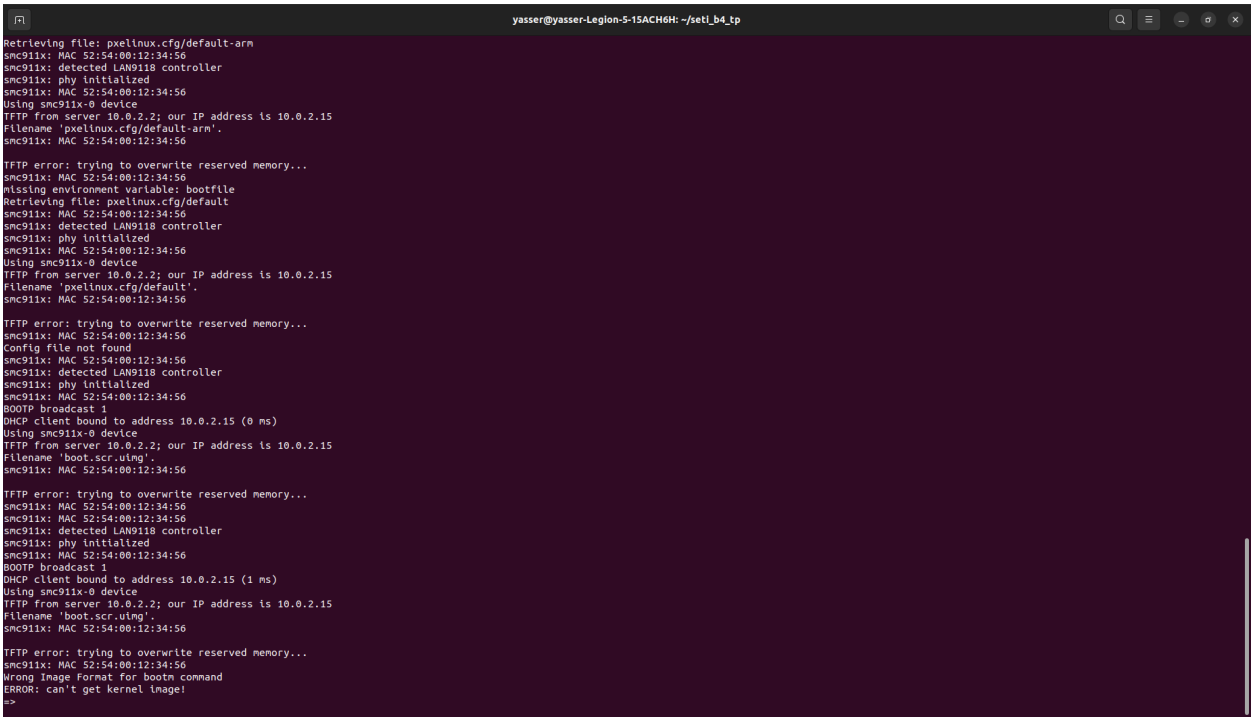


Figure 6: Execution of BusyBox image on QEMU.

1.7 Complete minimal system

For this part, we will be emulating the ARM core with a full system, that is, the kernel image, the **initramfs** image(which contains the filesystem) and the device tree.

So far, **QEMU** has been acting as the bootloader for our linux kernel, we will now use the **U-boot** bootloader to load the aforementioned images, of course, the bootloader also has to be executed and loaded into a machine (in our case, a virtual one emulated by **QEMU**), on a real machine, this will be done by the preloader.

In the previous sections, we have created the different images for a complete kernel boot, now is the step to put all these different components into one device (in our case, an **SD Card** image that will be loaded by **QEMU**). We first start by creating a file that contains the **SD Card** image:

```
$ mkdir sdcard
$ cd sdcard
$ dd if=/dev/zero of=sd bs=1M count=64
```

Then, we have to create a partition for this **SD Card**. We will only need one **FAT** partition covering the whole card for this application. To do that, we launch `parted` and create label and partition itself:

```
(parted) mklabel msdos
(parted) mkpart primary fat32 1MiB 100%.
```

Then we have to create a virtual disk partition on our machine via

```
$ sudo losetup -f --show -P sd.
```

This command is used to make the partition previously created appear as a device in **/dev**. It also prints the partition name, in my case it was `/dev/loop15`. This will be helpful later in creating the file system in the **SD Card**.

Further, we need a FAT filesystem on our SD card, so we create and mount it our virtual drive:

```
$ sudo mkfs.fat -F 32 /dev/loop0p1
$ mkdir mnt
$ sudo mount /dev/loop15p1 mnt.
```

After having created the file system in the partition, we copy the three components needed for a full kernel boot, that is:

```
$ sudo cp ../linux-5.15.6/build/arch/arm/boot/zImage mnt
$ sudo cp ../linux-5.15.6/build/arch/arm/boot/dts/vexpress-v2p-ca9.dtb mnt
$ sudo cp ../initramfs-busybox/initramfs.gz mnt
```

The first command copies the kernel image into the **SD Card**. The second is for the device tree, lastly, the third is for the **initramfs** image.

We then have to encapsulate the **initramfs** image by a special **U-Boot** header, this is essential, as it converts the image into a **u-boot** file that is recognized by it. The following command is used:

```
$ sudo ../../u-boot-2020.10/tools/mkimage \
-A arm -O linux -T ramdisk \
-d initramfs.gz uinitramfs
```

Finally we are able to run **QEMU** with **U-Boot**, and also integrating the **SD Card** into the emulated machine:

```
$ qemu-system-arm -machine vexpress-a9 \
-nographic -kernel u-boot-2020.10/u-boot -sd sdcard/sd
```

After **QEMU's** boot, we use **U-Boot** to load the contents of the **SD Card** into memory (specifically, **0x62000000, 0x63000000, 0x63100000** as magic addresses):

```
$ fatload mmc 0:1 0x62000000 zImage
$ fatload mmc 0:1 0x63000000 vexpress-v2p-ca9.dtb
$ fatload mmc 0:1 0x63100000 uinitramfs
```

Now that all the pieces needed for a full kernel boot are ready, we can launch the kernel using the command:

```
$ bootz 0x62000000 0x63100000 0x63000000
```

This command, evidently, boots the kernel using the 3 memory addresses given as arguments. These memory addresses now contain the three pieces needed for the kernel boot: `zImage`, the `DTB`, and the `initramfs`. After typing the last command, Linux kernel is correctly launched, as it can be seen on the image 7.

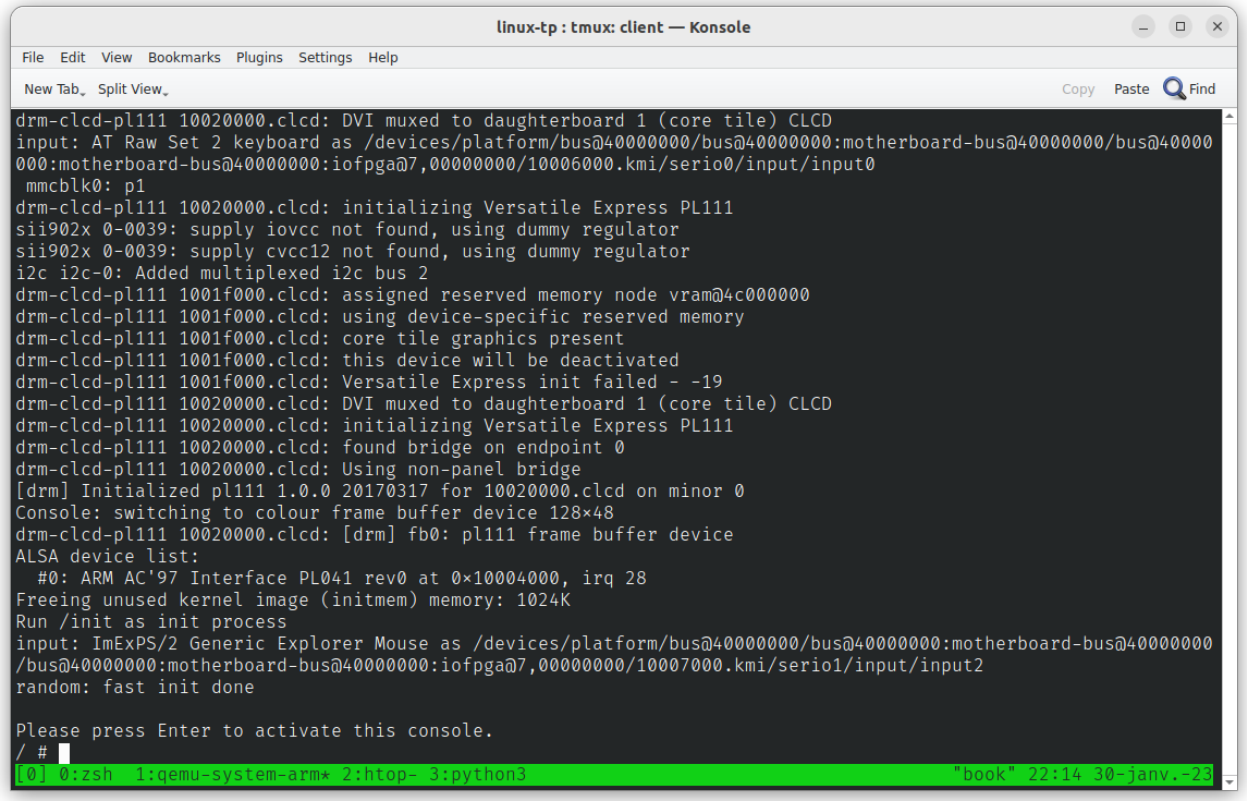


Figure 7: U-Boot BusyBox Linux launch

1.8 Conclusion

In this first TP, we got to understand what is really a kernel boot, and what is needed for a complete and successful kernel boot. We also got to explore concepts such as machine emulation and cross-compiling programs for the target machine.

The first attempts have shown us that a kernel cannot fully launch without a full system environment consisting of a filesystem, usually included in the **initramfs** and `init` program on that filesystem.

We then got to use the **BusyBox** framework, that allows for a fast setup of an embedded **Linux** ecosystem. At the end, we have shown that we can setup our own embedded system containing the kernel image and **initramfs** and load it into **QEMU**, using **Das U-boot** bootloader.

2 TP2: First driver

2.1 Intro

The aim of this TP is to understand how drivers and devices in Linux systems are working, by creating the driver from scratch. As a device, we are targeting to use a rather simple I^2C chip – **ADXL345** accelerometer. This accelerometer could be accessed via I^2C bus, presented on the board and simulated by QEMU, which allows us debugging our driver code without an access to the physical one.

We are targeting to create a driver for it, that will allow us to get the accelerometer measurements and set the specific settings of it.

2.2 Prerequisites

Since we used a bare Linux and built it from scratch during the last TP, the compiled kernel doesn't know that such a device exists. It requires us to download a version of Linux that includes a proper board device tree, at the very end of which a mention of **ADXL345** could be found:

```
&v2m_i2c_dvi {
      adxl345: adxl345@53 {
          compatible = "qemu,adxl345";
          reg = <0x53>;
          interrupt-parent = <&gic>;
          interrupts = <0 50 4>;
      };
};
```

From this part it could be seen that our drive must be `"qemu,adxl345"` compatible, the device uses one of two available addresses (`0x1D` and `0x53`). Moreover, the interrupts are routed to the `gic` interrupt controller of Cortex-A9.

As we mentioned before, we are using a patched QEMU simulator to have a virtual accelerometer, to be able to debug the code without a physical board. It will emulate the behavior of the accelerator and allow to update the registers via I^2C bus.

2.3 Driver skeleton

As a point of start, we are taking the skeleton, presented in the listing 2. As a first change we are adding some debug prints to the `*_probe` and `*_remove` functions, responsible for connection and disconnection of the device.

In this file we can also observe two device tables, registered with `MODULE_DEVICE_TABLE` macro. The first one, `ADXL345_idtable` is used with the kernel built-in devices and is considered as legacy. The second one, `ADXL345_of_match`, is used with Device Trees, and `of` stands for **Open Firmware**. Similar device tables allow to enumerate hot-plugged devices, which is quite helpful in USB drivers.

Another macro used in that code snippet, namely `module_i2c_driver(...)` is used to create initialisation and de-registration functions for the driver.

The last three macros names are saying for themselves: they define license type, module and author names.

We are going to use the simple makefile that will produce a `*.ko` driver module. It just need some header files, taken from the Linux source files. Its content is presented in the listing 3.

```c
#include <linux/init.h>
#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/of.h>
#include <linux/i2c.h>

static int ADXL345_probe(struct i2c_client *client,
const struct i2c_device_id *id) {
    // TODO
}
static int ADXL345_remove(struct i2c_client *client) {
    // TODO
}

static struct i2c_device_id ADXL345_idtable[] = {
    { "ADXL345", 0 },
    { }
};

MODULE_DEVICE_TABLE(i2c, ADXL345_idtable);
#ifdef CONFIG_OF
static const struct of_device_id ADXL345_of_match[] = {
    { .compatible = "qemu,ADXL345",
    .data = NULL },
    {}
};

MODULE_DEVICE_TABLE(of, ADXL345_of_match);
#endif
static struct i2c_driver ADXL345_driver = {
    .driver = {
        .name = "ADXL345",
        .of_match_table = of_match_ptr(ADXL345_of_match),
    },
    .id_table = ADXL345_idtable,
    .probe = ADXL345_probe,
    .remove = ADXL345_remove,
};

module_i2c_driver(ADXL345_driver);

MODULE_LICENSE("GPL");
MODULE_DESCRIPTION("ADXL345 driver");
MODULE_AUTHOR("Me");
```

Listing 2: Driver skeleton

The driver can be built using the following command:

```
$ make CROSS_COMPILE=arm-linux-gnueabihf- ARCH=arm\
KDIR=../linux-5.10.19/build/
```

In any case, it needs to do something. So let's add some prints to know that our driver starts.

```
ifneq (£(KERNELRELEASE),)
# kbuild part of makefile
obj-m := adxl345.o
else
# normal makefile
KDIR ?= /lib/modules/`uname -r`/build
default:
        $(MAKE) -C $(KDIR) M=$$PWD
endif
```

Listing 3: Driver makefile

2.4 Making the driver more useful

Sadly we are not allowed to use ordinary `printf` to output information. That's because we are in the kernel space and driver is not an ordinary process, so don't have famous `stdin`, `stdout`, `stderr` file descriptors attached. Instead, we have to use `printk(...)` function, which will output in the system log. A modified driver functions are presented in the listing 4.

To see the `print` output, we can pass by `dmesg`, provided by **Busybox**, or just update the log level to see the message instantly – with level 8 any type of message appears to terminal output:

```
$ echo 8 > /proc/sys/kernel/printk.
```

To see what's happens when we load the driver, we have to compile it first, and then use the `adxl345.ko` file.

```
static int ADXL345_probe(struct i2c_client *client,
const struct i2c_device_id *id){
    printk(KERN_INFO "ADXL345 connected\n");
    return 0;
}
static int ADXL345_remove(struct i2c_client *client){
    printk(KERN_INFO "ADXL345 disconnected\n");
    return 0;
}
```

Listing 4: Driver modified functions

After compilation, we start the emulator with

```
$ ./qemu-system-arm -nographic -machine vexpress-a9 \
-kernel linux-5.10.19/build/arch/arm/boot/zImage \
-dtb linux-5.10.19/build/arch/arm/boot/dts/vexpress-v2p-ca9.dtb \
-initrd rootfs.cpio.gz \
-fsdev local,path=driver_files,security_model=mapped,id=mnt \
-device virtio-9p-device,fsdev=mnt,mount_tag=mnt
```

Note that we include a folder with compiled driver, adding a virtual device.

After booting, we have to enable the device with filesystem to make `/mnt` available, using the specific config for blocksize and version:

```
$ mount -t 9p -o trans=virtio mnt /mnt -oversion=9p2000.L,msize=10240
```

With all above, we can finally load and unload the module with our driver via

```
$ insmod /mnt/adxl345.ko
$ rmmod adxl345.
```

The result of the execution is presented in figure 8.

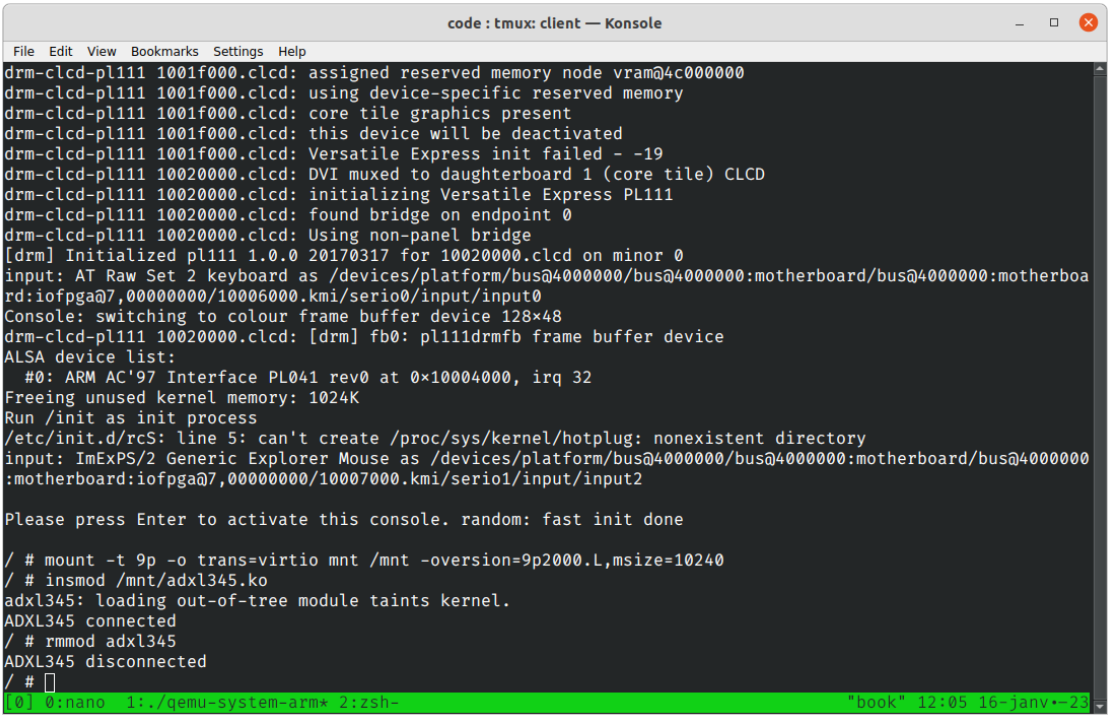


Figure 8: First driver messages

As expected, we see the messages from `probe` and `remove` functions, because the device is present in the system, so these functions are executed on driver load/unload.

However, we see an unexpected message too: the kernel is tainted for some reason. This happens because our driver is compiled outside of device tree. Anyway, it is just a warning, let's skip it for now and pass to I^2C messages.

```
typedef enum {
    ADXL345_DEVID_REG =        0x00,
    ADXL345_BW_RATE_REG =      0x2C,
    ADXL345_DATA_FORMAT_REG =  0x31,
    ADXL345_FIFO_CTL_REG =     0x38,
    ADXL345_POWER_CTL_REG =    0x2D,
    ADXL345_INT_ENABLE_REG =   0x2E,
    ADXL345_INT_MAP_REG =      0x2F,
    ADXL345_DATAX0_REG =       0x32,
    ADXL345_DATAX1_REG =       0x33,
    ADXL345_DATAY0_REG =       0x34,
    ADXL345_DATAY1_REG =       0x35,
    ADXL345_DATAZ0_REG =       0x36,
    ADXL345_DATAZ1_REG =       0x37,
} adxl_reg_t;
```

Listing 5: `adxl345.h` header file

2.5 ADXL345 first I²C interaction

Firstly, we would like to know the value of the ID register, which is always `0xE5`. This will help us to know if the bus is working and the devices can receive our commands.

For those purposes, a small header is written, it is presented in the listing 5. It will be updated with the addition of register addresses.

To the main driver file, the `i2c_transfer` call was added, as presented in the listing 6. Of course, two separate calls to `i2c_master_send` and `i2c_master_recv` could be used, but it results in two transaction for the reading of single register. This does not undergo I²C philosophy with the special restart condition. To respect this, a transfer function is used.

The result of the code execution is presented in the figure 9. As it can be seen, the driver reads `0xE5`, which means it reads the register correctly.

Now it's time to move further, setting up the accelerometer parameters to be able to read its actual values.

2.6 ADXL345 setup

For the initial ADXL setup, we are to modify certain registers, to set the different sensor parameters as mentioned in the TP. This is performed via the `ADXL_345_setup` function from the listing 8. It uses setter and getter, described in the listing 7.

As a result of configuration, we obtain the terminal output, shown in figure 10.

At this point we are sure that our write, read, setup functions are working correctly, causing no errors. We then can proceed with the driver development in the next TP.

2.7 Conclusion

In this TP, we got to see how we can add device drivers to our kernel, in particular, we have created an **ADXL345** driver, able to interact with the respecting accelerometer.

```c
static int ADXL345_probe(struct i2c_client *client,
const struct i2c_device_id *id) {
    printk(KERN_INFO "ADXL345 connected\n");

    adxl_reg_t reg_id = ADXL345_DEVID_REG;
    uint8_t data;
    struct i2c_msg msg[2] = {
        { .addr = client->addr, .len = 1, .buf = (uint8_t*) &reg_id },
        { .addr = client->addr, .len = 1, .buf = &data, .flags = I2C_M_RD }
    };

    int ret = i2c_transfer(client->adapter, msg, 2);

    if (ret < 0) {
        printk(KERN_WARNING "ADXL345 is not responding, error %d\n", ret);
        return ret;
    }

    printk(KERN_INFO "ADXL345 DEVID value is 0x%X\n", data);
    return 0;
}
```

Listing 6: Update of probe function

```c
static int ADXL345_read_reg(struct i2c_client *client,
    adxl_reg_t reg, uint8_t* data) {
    struct i2c_msg msg[2] = {
        { .addr = client->addr, .len = 1, .buf = (uint8_t*) &reg },
        { .addr = client->addr, .len = 1, .buf = data, .flags = I2C_M_RD }
    };

    return i2c_transfer(client->adapter, msg, 2);
}

static int ADXL345_write_reg(struct i2c_client *client,
    adxl_reg_t reg, uint8_t* data) {
    uint8_t transfer[2] = {reg, *data};
    return i2c_master_send(client, transfer, 2);
}
```

Listing 7: Read and write register functions

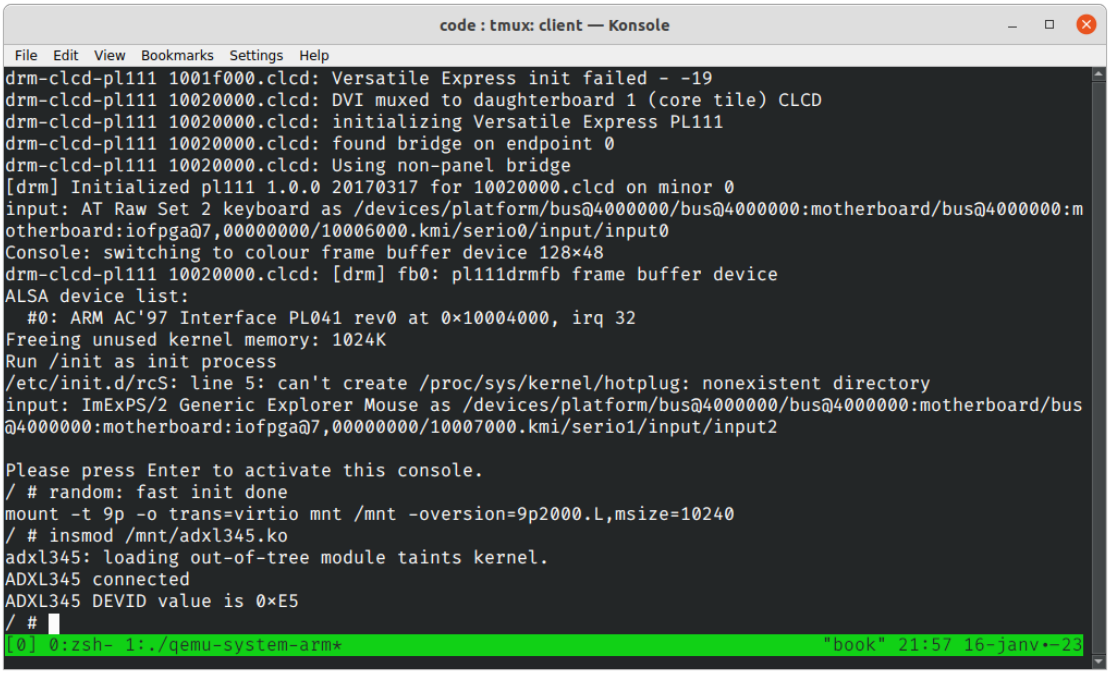


Figure 9: Result of DEVID register read

We have also gotten a chance to use the different I^{2}C functions such as `i2c_transfer()` and `i2c_msg()`. These functions exist thanks to the lower-level API of the Linux (HAL), and we don't have to manipulate the registers by hand to perform a data transfer. We then used those functions to check the link between the driver and the device by verifying its **DEV_ID** register, and then initialize the device by modifying its different registers and successfully reading them back.

```c
static int ADXL345_setup(struct i2c_client *client) {
    adxl_reg_pair_t setup[5] = {
        { .reg = ADXL345_BW_RATE_REG, .value = 0x0A },
        { .reg = ADXL345_INT_ENABLE_REG, .value = 0 },
        { .reg = ADXL345_DATA_FORMAT_REG, .value = 0 },
        { .reg = ADXL345_FIFO_CTL_REG, .value = 0 },
        { .reg = ADXL345_POWER_CTL_REG, .value = 1 << 3 },
    };

    for (int i = 0; i < 5; ++i) {
        if (ADXL345_write_reg(client, setup[i].reg, &(setup[i].value)) < 0) {
            break;
        }
    }

    return ret;
}
```

Listing 8: Setup function

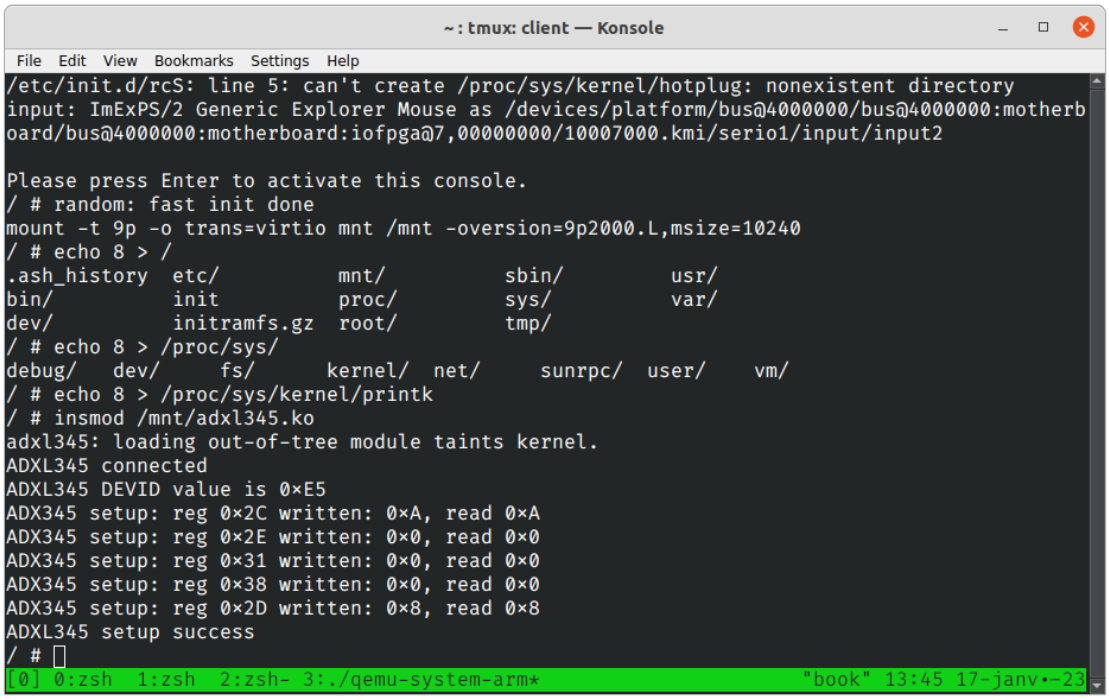


Figure 10: Result of accelerometer setup

```c
static int ADXL345_probe(struct i2c_client *client,
const struct i2c_device_id *id) {
    printk(KERN_INFO "ADXL345 connected\n");

    uint8_t data;
    int ret = ADXL345_read_reg(client, ADXL345_DEVID_REG, &data);

    if (ret < 0){
        printk(KERN_WARNING "ADXL345 is not responding, error %d\n", ret);
        return ret;
    }

    printk(KERN_INFO "ADXL345 DEVID value is 0x%X\n", data);

    ret = ADXL345_setup(client);

    if (ret < 0){
        printk(KERN_WARNING "ADXL345 setup error %d\n", ret);
        return ret;
    }

    printk(KERN_WARNING "ADXL345 setup success\n");
    return 0;
}

static int ADXL345_remove(struct i2c_client *client) {
    uint8_t data = 0;
    int ret = ADXL345_write_reg(client, ADXL345_POWER_CTL_REG, &data);

    if (ret < 0) {
        printk(KERN_WARNING "ADXL345 is not responding, error %d\n", ret);
        return ret;
    }

    printk(KERN_INFO "ADXL345 disconnected\n");
    return 0;
}
```

Listing 9: Final version of probe and disconnect functions

3 TP3: Driver access from user space

3.1 Intro

During this TP we are going to fulfill the dedication of our ADXL345 driver, described in the previous part: make its data accessible to the user of our Linux system. In addition, we are going to make the driver configuration accessible by the user, allowing him to change some parameters, e. g. measure frequency or axis measured.

To do that, we are going to use the **Misc** framework, allowing us to create a device pseudo file. Of course, we are using our device as a character one, and we are profiting of Misc framework, because we do not have to define the major version (it is always 10), and using a dynamic minor.

3.2 Misc framework description

The misc framework is a lightweight framework, it simplifies the management of special files, placed usually in the `/dev` folder. You can overload different file operations like read, write, open ans release but there are two major differences to a character framework.

The first one is the misc device always has the major device number 10 and on character device you can use the major device number more freely. Also the misc device it is much easier to set up than the character device, because the principal functionality of pseudo file is already done by framework, it leasts only to set up the callbacks on needed system calls, and that's it.

3.3 Misc framework in our driver

The point of start is just creating the device-representing structure, for now it will include only the structure `miscdevice`:

```
struct adxl345_device {
    struct miscdevice miscdev;
};
```

Since the structure of device is defined, we have to update the `*_probe()` function. The updated part is presented in the listing 10. We dropped the error code treatment and debug prints in this snipped to increase readability. In fact, the code does following things:

- Allocate the memory for the device structure
- Allocate the memory and print filename in it, which go to the `/dev/*`
- Set the misc driver parameters: dynamic minor, name, parent (which is I^2C device actually), and structure of available operations.
- Register backwards our device as a low-level one for the I^2C
- Finally register the misc device.

The same manner, `*_release()` function was updated, to ensure that the driver cleanup after himself: kernel memory leakage is too dangerous!

Updated part is presented in the listing 11. Mainly it just retrieves the actual devices and do two `kfree()` to free the memory, allocated before.

```
    struct adxl345_device* dev =
        kmalloc(sizeof(struct adxl345_device), GFP_KERNEL);
    char* name_buf = kasprintf(GFP_KERNEL, "adxl345-%d", dev_count);

    /*...*/

    dev->miscdev.minor = MISC_DYNAMIC_MINOR;
    dev->miscdev.name = name_buf;
    dev->miscdev.parent = &(client->dev);
    dev->miscdev.fops = &fops;

    i2c_set_clientdata(client, dev);
    int misc = misc_register(&(dev->miscdev));

    /*...*/

    dev_count += 1;
```

Listing 10: `ADXL345_probe()` function update

```
    dev_count -= 1;
    struct adxl345_device* dev = i2c_get_clientdata(client);

    misc_deregister(&(dev->miscdev));
    kfree(dev->miscdev.name);
    kfree(dev);
```

Listing 11: `ADXL345_release()` function update

3.4 Test program

To test the driver, one could write a simple C program with `open` and `read` functions.

In fact, the read could be done via built-in **BusyBox** utilities: `dd` and `xxd`:

```
dd bs=2 count=1 if=/dev/adxl345-0 | xxd.
```

This code reads 2 bytes from the pseudo file, since each measurement contents 16-bit 2's complement value. Moreover, it should be run in a loop with 0.01s, or 10 ms delay, since the data rate is set to 100 Hz.

So some kind of following code snippet should work:

```
while :
  do
    dd bs=2 count=1 if=/dev/adxl345-0 status=none | xxd
    sleep 0.01
  done
```

That does not always provide coherent results: sometimes the driver reads axis measurements from different iterations, which is quite understandable since we are running asynchronous code without any interruptions. This could be observed as signs on the values obtained for X, Y, and Z axes. Obtained output is presented in figure 11. Note that we don't really interested in the debug print, but in the `xxd` output.

Normally, we already ensured that our driver returns valid values, but to make sure we also wrote a simple C program to test the system calls. Its contents is presented in listing 12. As usual, it should be complied with static library linking and put in the driver folder, to be accessed in `/mnt` in simulated system.

```
typedef enum {
    ADXL345_AXIS_X = 0,
    ADXL345_AXIS_Y = 2,
    ADXL345_AXIS_Z = 4
} adxl_axis_t;

int msleep(unsigned int tms) {
  return usleep(tms * 1000);
}

int main(){
    int fd, ret;
    fd = open("/dev/adxl345-0", O_RDWR);
    /*...*/

    char buffer[2];
    const int count = 2;

    for (int i = 0; i < count; i++) {
        ret = read(fd, buffer, 2);
        short actual_value = buffer[1] << 8 | buffer[0];
        printf("Read value is : %hi\n", actual_value);

        msleep(40);
    }

    /*...*/
    return 0;
}
```

Listing 12: Driver test code

At this point we could be sure that our driver is working correctly and outputs the right values from the registers. It is left only configuring the axis that should be read. This should be done by the second callback that we are going to write: the **ioctl** one.

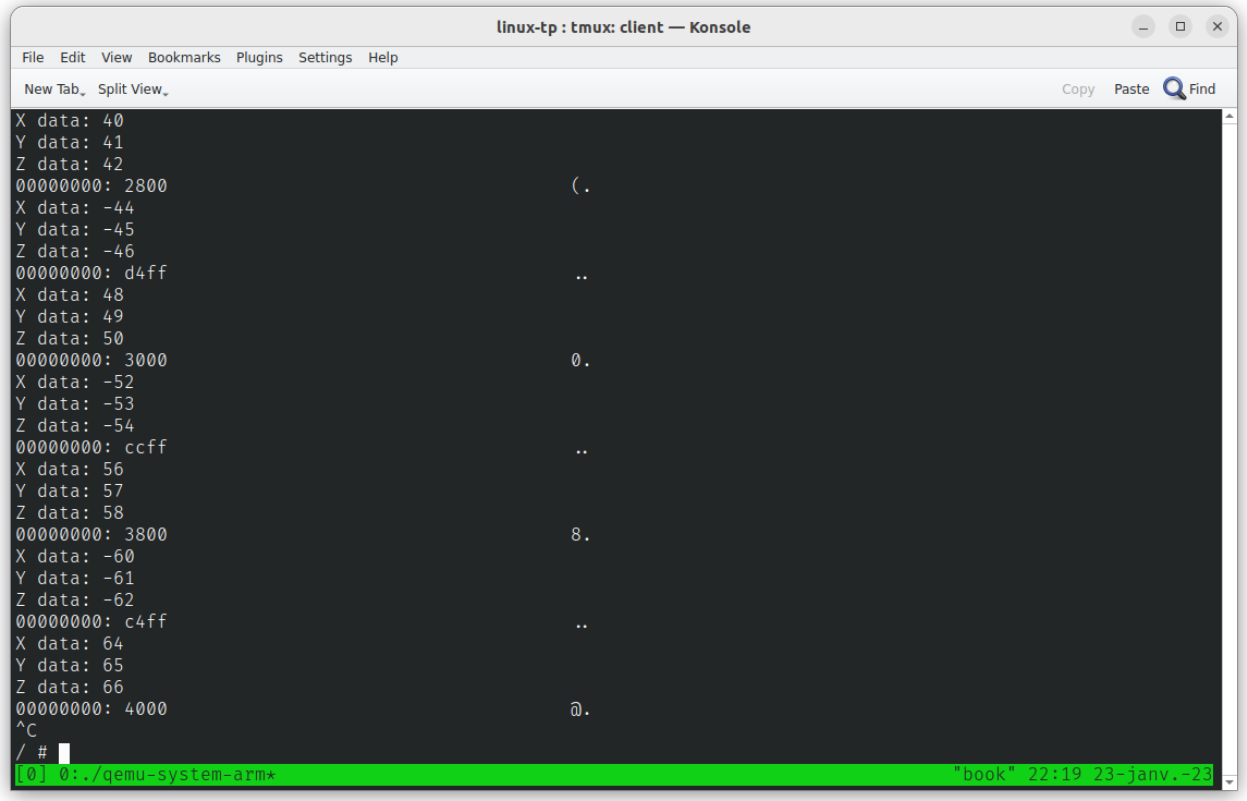


Figure 11: ADXL345 X-axis output

3.5 Calls to `ioctl`

`ioctl`, as an abbreviated of input-output control, is a most primitive way to configure the drivers and/or devices from the user space. Basic idea is rather simple: by this call, we can pass one command of 32 bit (less in fact) and its parameter as a pointer. Thanks to this architecture, almost every needed procedures could be implemented, except for the bandwidth-hungry ones, such as video card drivers. At least I don't see a problem writing a printer driver with `ioctl`.

First thing first, we have to understand why our command in fact is smaller than expected. This is due to coding convention and macro usage. Two bits of the command is dedicated to understand its type – **read**, **write**, **read-write**. These 2 bits are followed by the argument size (14 bits). Additional 8 bits is describing the major of the device driver (10 in our case) that will treat the call. As a result, command is 8-bits only. This small variety of commands is corrected by the ability to pass a buffer from the user space to the kernel space.

For our purposes it seems like an only purpose of such call will be setting an axis of accelerometer to get its measurements. Thus, I've added three commands for each axis to be sent by the driver. The realisation is presented in the listing 13. It is a simple function that just update the global variable on respecting command, which is then used in `*_read(...)` function to retrieve selected axis measurement.

The test program was also modified to change the axis for different measurements. Its final code is presented in the listing 14. As a result, driver could successfully change the axis to output its measurements, as it can be seen in the image 12.

```
long adxl345_ioctl(struct file* file, unsigned int cmd, unsigned long arg) {
    printk(KERN_INFO "Command data: %x\n", cmd);

    switch (cmd) {
        case _IOW(MISC_MAJOR, ADXL345_AXIS_X, uint8_t):
            axis = (adxl_axis_t) ADXL345_AXIS_X;
            return 0;
        case _IOW(MISC_MAJOR, ADXL345_AXIS_Y, uint8_t):
            axis = (adxl_axis_t) ADXL345_AXIS_Y;
            return 0;
        case _IOW(MISC_MAJOR, ADXL345_AXIS_Z, uint8_t):
            axis = (adxl_axis_t) ADXL345_AXIS_Z;
            return 0;
        default:
            printk(KERN_WARNING "ADXL345 illegal ioctl command: %d\n", cmd);
            return -EINVAL;
    }
}
```

Listing 13: `adxl345_ioctl()` function

3.6 File isolation for different processes

Everything is fine with the code of driver, except the fact that if acelerometer is used by different programs, they could change the measurement axis without knowledge of each others. It worth making an independent choice, that means that our driver have to remember which process asked for which axis measurement.

To do so, we don't have a lot of choices: such distinguish of processes could be done via variable `current`, that contains `pid` of the process. We are going to store its pid with the selected axis in the linked list. Every read of accelerometer will read all three axis, and returning the value selected by currennt process.

To do so, it worth adding some basic operations with the linked list, as

- `add_subscriber(sub)`
- `remove_subscriber(sub)`
- `find_subscriber(pid)`
- `last_subscriber()`.

This list is managed in the `*_open()` and `*_release()` functions, to add and remove process pids with their settings.

The result of execution of the same test program is presented in figure X.

It can be seen that actual file release happens not at the end of the program, but at its startup. Actually it doesn't matter for the kernel, when exactly it happens. The most important thing is that memory should be returned to the allocator.

However, our current test program contains only one thread, which is not good to test the current driver. Program need to start 2 or more threads at the same time to verify that axes are

```c
int main(){
    int fd, ret;
    fd = open("/dev/adxl345-0", O_RDWR);
    /*...*/
    adxl_axis_t testcase[3] = {ADXL345_AXIS_X, ADXL345_AXIS_Y, ADXL345_AXIS_Z};

    for (int i = 0; i < 3; i++) {
        ret = ioctl(fd, _IOW(10, testcase[i], uint8_t));
        /*...*/

        char buffer[2];
        const int count = 2;

        for (int j = 0; j < count; j++) {
            ret = read(fd, buffer, 2);
            short actual_value = buffer[1] << 8 | buffer[0];
            printf("Read value is : %hi\n", actual_value);

            msleep(40);
        }
    }
}
```

Listing 14: Driver test code with `ioctl`

selected correctly. This could be done by the `fork()` function, updated test program snippet is presented in the listing 16.

As could be seen in the image 14, the driver works as expected, giving different axis to the different processes.

However, it seems like it is not a casual way to give processes proper information. It seems like more proper approach is to use single process (daemon) in user space that connects to the driver and retrieve all needed information from it. The same way it collects the information from the client processes, which may connect to the daemon however they want, with pipes, sockets, files... The improvements of that approach is that we are not creating potentially dangerous leaking structures in kernel space, but in user one. In the worth case scenario, kernel would kill the daemon because of segmentation fault, but kernel would remain consistent. In current approach, if something goes wrong in driver – kernel sinks with it.

3.7 Conclusion

In this section we are learned how to use the **Misc** framework and how it eases driver development. We added read function to our driver, allowing different processes to read the measurements from the pseudo file in `/dev`. Moreover, we added driver control via `ioctl()` call, allowing processes to choose the axis of measurements.

As a final step, we separated measurements for different processes, that way that `ioctl()` of one process does not affect others.

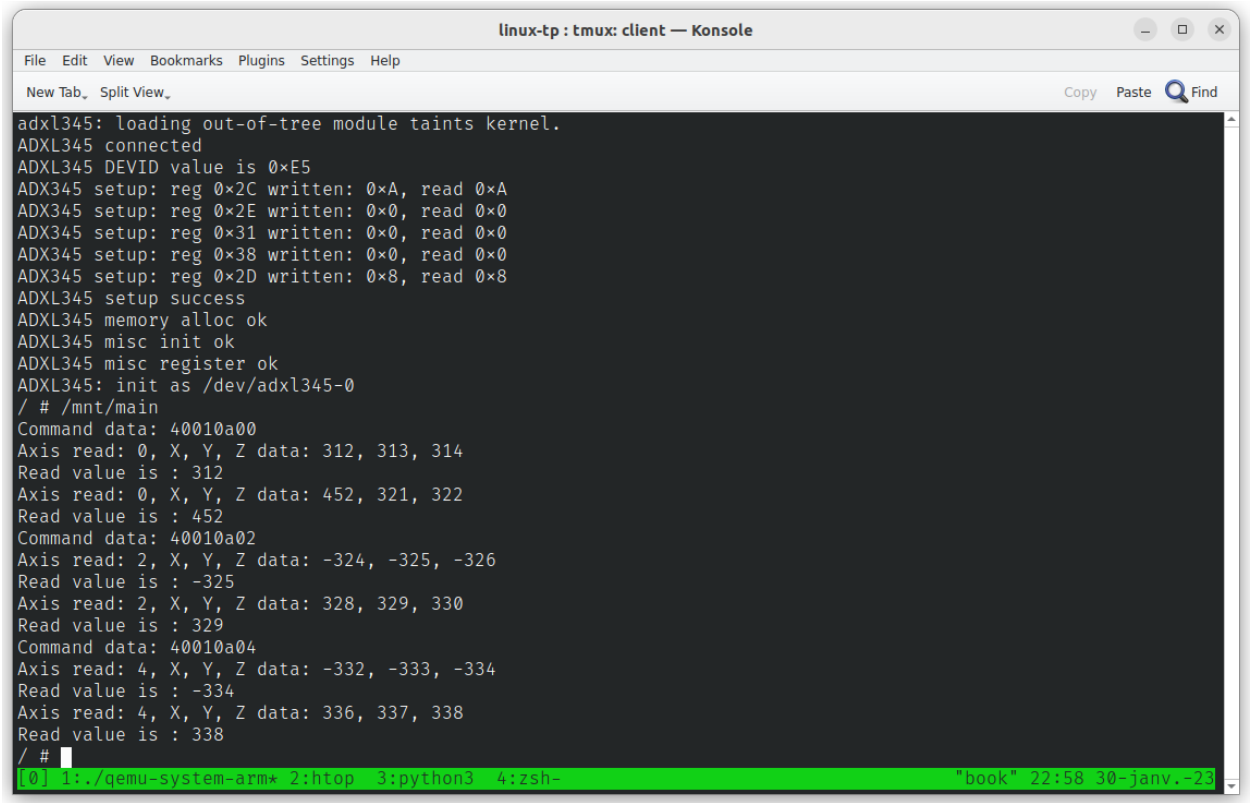


Figure 12: Calls to `ioctl`

```c
int adxl345_open(struct inode * inode, struct file * file) {
    pid_t pid;
    struct adxl_association_s* assoc;

    pid = current->pid;

    printk(KERN_INFO "Open by %d", pid);

    assoc = kmalloc(sizeof(struct adxl_association_s), GFP_KERNEL);
    assoc->pid = pid;
    assoc->axis = ADXL345_AXIS_X;
    assoc->next = NULL;

    printk(KERN_INFO "Adding in %d", pid);
    add_subscriber(assoc);

    return 0;
}

int adxl345_release(struct inode * inode, struct file * file) {
    pid_t pid;
    struct adxl_association_s* sub;

    pid = current->pid;
    sub = find_subscriber(pid);

    printk(KERN_INFO "Release by %d", pid);

    /*...*/

    remove_subscriber(sub);
    kfree(sub);

    return 0;
}
```

Listing 15: Driver `open` and `release`

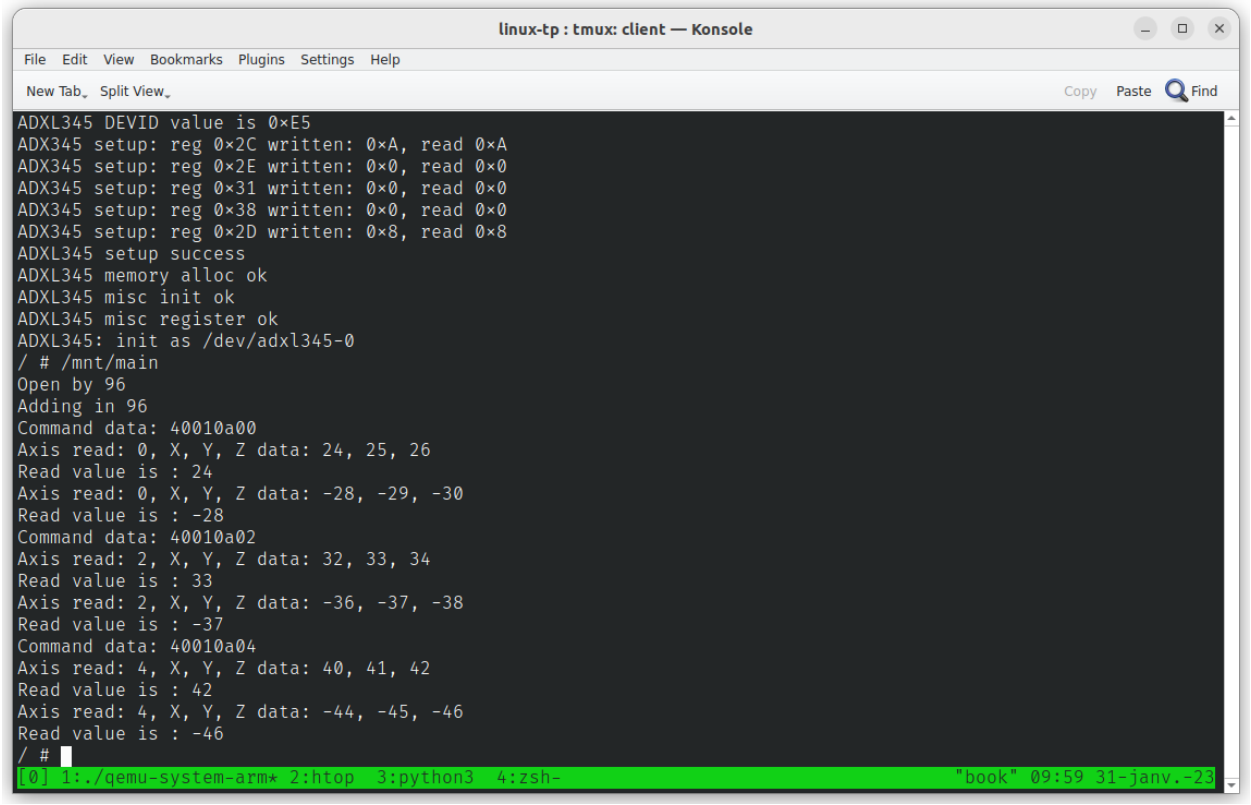


Figure 13: Driver with memorization request

```c
/*...*/
pid_t pid = fork();

int fd = open("/dev/adxl345-0", O_RDWR);
if (fd < 0) {
    printf("Something went wrong\n");
    return -1;
}

int ret;
adxl_axis_t testcase_p[3] = {ADXL345_AXIS_X, ADXL345_AXIS_Y, ADXL345_AXIS_Z};
adxl_axis_t testcase_c[3] = {ADXL345_AXIS_Y, ADXL345_AXIS_X, ADXL345_AXIS_Z};

adxl_axis_t* testcase = (pid == 0) ? testcase_c : testcase_p;
/*...*/
```

Listing 16: Driver test code with fork

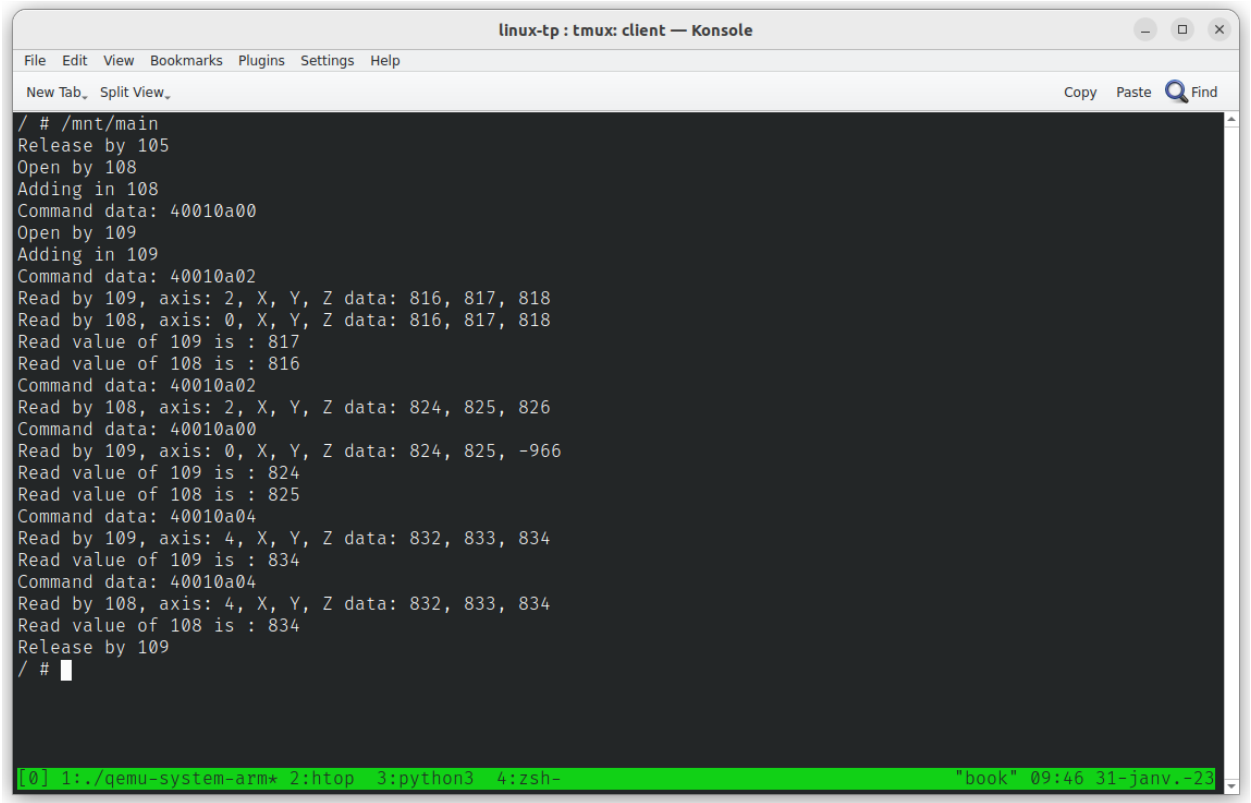


Figure 14: Different measures for different processes

4 TP4: Interrupts

4.1 Intro

This part of the driver development is concerned with the interrupts.

We are going to fulfill this last contract of any type of embedded peripheral and will see how the interrupts are treated by the Linux kernel. The final goal consists of the following steps:

- Changing the mode of **ADXL345** FIFO
- Enabling the interrupts on the accelerometer side
- Creation and registration of the interrupt handler.

4.2 Update of the ADXL345 setup

In the previous parts, we were using the same setting for the accelerometer: no FIFO and no interruptions.

We just consecutively read the registers of values on X, Y, and Z values and returned them to the application, which could introduce an inconsistency in the data, because we didn't control its flow. In fact, the application requesting the values of acceleration could read the same value n times without noticing it, because the driver didn't allow the application to know about the rate of the incoming data.

To avoid such an inconsistency, we are enabling FIFO and *waterfill* interrupt on it. This means that the accelerometer will accumulate the data in the internal FIFO (of 32 records for each axis), and generate the interrupt when the number of measurements overpasses the water flow limit. To achieve that, we slightly modified the initial register values, as it is shown in the listing 17. Here we are enabling the needed interrupt in the `INT_ENABLE_REG`, then `FIFO_CTL_REG` is updated to allow `streaming mode` (continue to fill the FIFO even it's full, overwriting the old data), with the watermark level of 20 entries.

```
adxl_reg_pair_t setup[5] = {
        { .reg = ADXL345_BW_RATE_REG, .value = 0x0A },
        { .reg = ADXL345_INT_ENABLE_REG, .value = 1 << 1 },
        { .reg = ADXL345_DATA_FORMAT_REG, .value = 0, },
        { .reg = ADXL345_FIFO_CTL_REG, .value = 1 << 7 | WATERFILL_LIMIT },
        { .reg = ADXL345_POWER_CTL_REG, .value = 1 << 3 },
    };
```

Listing 17: ADXL345 registers setup

In addition, on the driver's side, we have to organize a FIFO too. This is necessary to economize the number of interruption calls. The idea is to wait for the waterfall interrupt, copy the data in the interrupt handler into the internal driver buffer, and then use it in the `*_read()` function. Thanks to this, we minimize the number of accesses to the accelerometer from the driver, allowing the treatment of other devices on the I^2C bus effectively. Otherwise, we would have to read each measurement in the `*_read()` function.

To implement this buffer, we are going to use the `kfifo` implementation, adding the corresponding field in the device structure, as shown in the listing 18. We set the size of the buffer to 64

```c
struct adxl345_measurement { uint16_t x; uint16_t y; uint16_t z; };

struct adxl345_device {
    struct miscdevice miscdev;
    struct wait_queue_head queue;
    DECLARE_KFIFO(fifo_samples, struct adxl345_measurement, 64);
};
```

Listing 18: ADXL345 struct update

elements, representing the measurements of 3 axes (6 bytes total). The size is dictated by the idea that the intermediate buffer should have a size at least twice larger than the waterfall limit. Due to the optimizations, `kfifo` size should be a power of 2, so instead of 40, we can choose only 64.

4.3 Interrupt handling

The idea of the interrupt handling and data passing to the userspace programs is including blocking the processes in the `*_read()` callback, while measurements are not available in the FIFO. This requires the usage of wait-like functions, which requires the waiting queue, added to the device struct in the listing 18.

```c
ssize_t adxl345_read(struct file * file,
    char __user * buf, size_t count, loff_t * f_pos){
    /* Subscriber identification is ommited */

    while (read_offset != count){
        if (kfifo_get(&(dev->fifo_samples), &el) == 0){
            wait_event_interruptible(dev->queue, data_is_available(dev));
            continue;
        }

        char* data_pointer = ((char*) &(el.x)) + (size_t) sub->axis;

        if (copy_to_user(buf + read_offset, data_pointer, 2)){
            return -EFAULT;
        }

        read_offset += 2;
    }

    return count;
}
```

Listing 19: `read` function update

First of all, the `*_read()` callback should be modified, as it is present in the listing 19. Basically, it's just checking the return value of `kfifo_get(...)` macro, which is 0 if nothing was read from

the FIFO. In the `wait_event_interruptible(...)` macro, a wrapper `data_is_avaliable(...)` is used, which is basically just a negation of `kfifo_is_empty(...)`, since wait macro does not accept another macro as a parameter.

As a result, the reading process will be blocked in the callback, until the data is available, which will be preceded by the `wake_up(...)` macro in the interrupt handler. Then, data is just copied by two bytes to the userspace.

We are going to use the Threaded IRQ mechanism to interrupt treatment since this approach allows to block/wait with no restrictions because it's done in the process of application. Since we are using the I^2C bus, which is relatively slow, all interactions should be preferably done in the bottom half of the interrupt, because during the interrupt handling, all other interrupts are disabled. In fact, in our driver we don't even need the top half of the handler, because the accelerometer measurement rate is slow too, so we only have to write the bottom part, since we already introduced the bufferisation to our driver. Of course, if we needed more reactivity of the driver, we would write the top half too, but the accelerometer measurement rate is only 100 Hz.

```
static int ADXL345_probe(struct i2c_client *client,
const struct i2c_device_id *id) {
    /* Setup is ommited */

    dev = kzalloc(sizeof(struct adxl345_device), GFP_KERNEL);
    name_buf = kasprintf(GFP_KERNEL, "adxl345-%d", dev_count);

    /* Misc framework init */

    i2c_set_clientdata(client, dev);

    misc = misc_register(&(dev->miscdev));

    init_waitqueue_head(&(dev->queue));

    ret = devm_request_threaded_irq(
        &(client->dev),
        client->irq,
        NULL,
        adxl345_irq,
        IRQF_ONESHOT,
        dev->miscdev.name,
        dev
    );

    printk(KERN_INFO "IRQ registered: %d\n", ret);

    return 0;
}
```

Listing 20: ADXL345 interrupt enable

In the listing 20 you can see the initialization of the interrupt: device is our I^2C one, interrupt number is retrieved via the I^2C client since it is a bit difficult to calculate it by hand, the top half interrupt handler is `NULL` since we don't use it, the bottom half function is `adxl345_irq()`, type of interrupt is `ONESHOT` (on the front of signal), name of the interrupt in the `proc/interrupts` is the same as the name of the device in the `/dev` folder.

Finally, the interrupt handler (bottom half) is presented in the listing 21.

```
irqreturn_t adxl345_irq(int irq, void* dev_id) {
    struct adxl345_device* dev = (struct adxl345_device*) dev_id;
    struct i2c_client* client = to_i2c_client(dev->miscdev.parent);

    size_t i;

    for (i = 0; i < WATERFILL_LIMIT; ++i) {
        int16_t buf[3];
        struct adxl345_measurement meas;

        ADXL345_read_axis(client, buf);

        meas.x = buf[0];
        meas.y = buf[1];
        meas.z = buf[2];

        kfifo_put(&(dev->fifo_samples), meas);
    }

    wake_up(&(dev->queue));

    return IRQ_HANDLED;
}
```

Listing 21: ADXL345 interrupt handler

The interrupt handler reads the 20 records from the FIFO of the accelerometer, since the interrupt is generated only if the amount of elements in the queue is set to 20, so at this point driver is safe. All the data is passed to the driver FIFO. At the end of the interrupt, we are calling `wake_up(...)` to resume the blocked in the `*_read(...)` callback, if present.

It worth note the following points: first, we are not protecting the queue, since it is always accessible for one concurrent read and write. Second, the data for the accelerometer should be read in the same transaction for all axes. This function was implemented in the previous parts, so we didn't have to debug it.

4.4 Test program

To test the new version of the driver, we added some debug prints to the interrupt, and slightly modified the previous test program. The debug print is surrounding the wait call, showing the periods of activity of the FIFO. In addition, one print is used to show the amount of free space in the FIFO, to know if it is working.

```
        printk(KERN_INFO "In FIFO: %d\n", kfifo_avail(&(dev->fifo_samples)));

        if (kfifo_get(&(dev->fifo_samples), &el) == 0){
            printk(KERN_INFO "Not enough data, sleeping\n");
            wait_event_interruptible(dev->queue, data_is_available(dev));
            printk(KERN_INFO "Data available, wake up\n");
            continue;
        }
```

```
int main(){
    int fd = open("/dev/adxl345-0", O_RDWR);
    /* Test the fd */

    for (int i = 0; i < 3; i++) {
        char buffer[2];
        const int count = 100;
        int ret = ioctl(fd, _IOW(10, 1 << i, uint8_t));

        /* Test the ioctl result */

        for (int j = 0; j < count; j++) {
            short actual_value = buffer[1] << 8 | buffer[0];
            printf("Read value of %d is : %hi\n", getpid(), actual_value);
        }
    }

    close(fd);
}
```

Listing 22: New test program

The test program code is presented in the listing 22. Basically, it does the same thing as before, except it is reading 100 values for each axis, to ensure that FIFO is being read several times.

As usual, we are running **QEMU** and installing the compiled module of the driver. Right after we check if the interrupt is correctly registered, we can execute the `cat /proc/interrupts` bash command, which output is presented in figure 15. As you can see, the kernel is registered correctly as the interrupt handler and added it to the list of the `CPU0`.

Finally, as usual, we are running our test program with `/mnt/main` in the terminal. The result of execution is presented in figure 16. As can be seen, the FIFO is being read correctly, with an increasing number of free slots, and sleep messages could be observed. Note that the difference in the number of free slots before and after waiting is not exactly 20, but 19, because the `kfifo_get(...)` pulls one element from the queue before the print of the free available space.

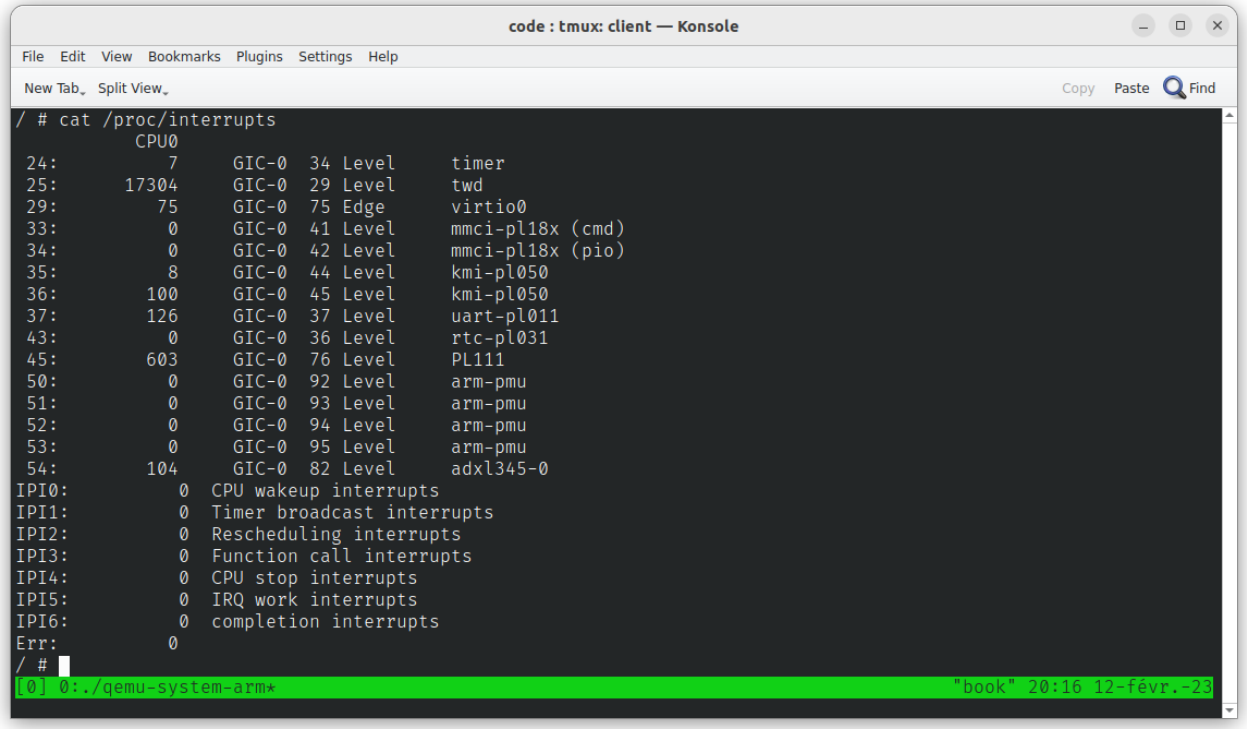


Figure 15: Registration of interrupt

4.5 Conclusion

In this part of the work, we have added the proper treatment of the data stream, coming from the accelerometer, using the interrupts. We have implemented the bottom half of *Threaded IRQ* to perform the I^2C transactions to read the FIFO of the accelerometer. We also used the `kfifo` module in the kernel to add data buffering on the driver level. Finally, we added a wait function for the `read` callback, which blocks the process, resumed by the `wake_up()` call in the interrupt handler, when new data is available, passing the data from the accelerometer to the user program at the rate of accelerometer measurements.

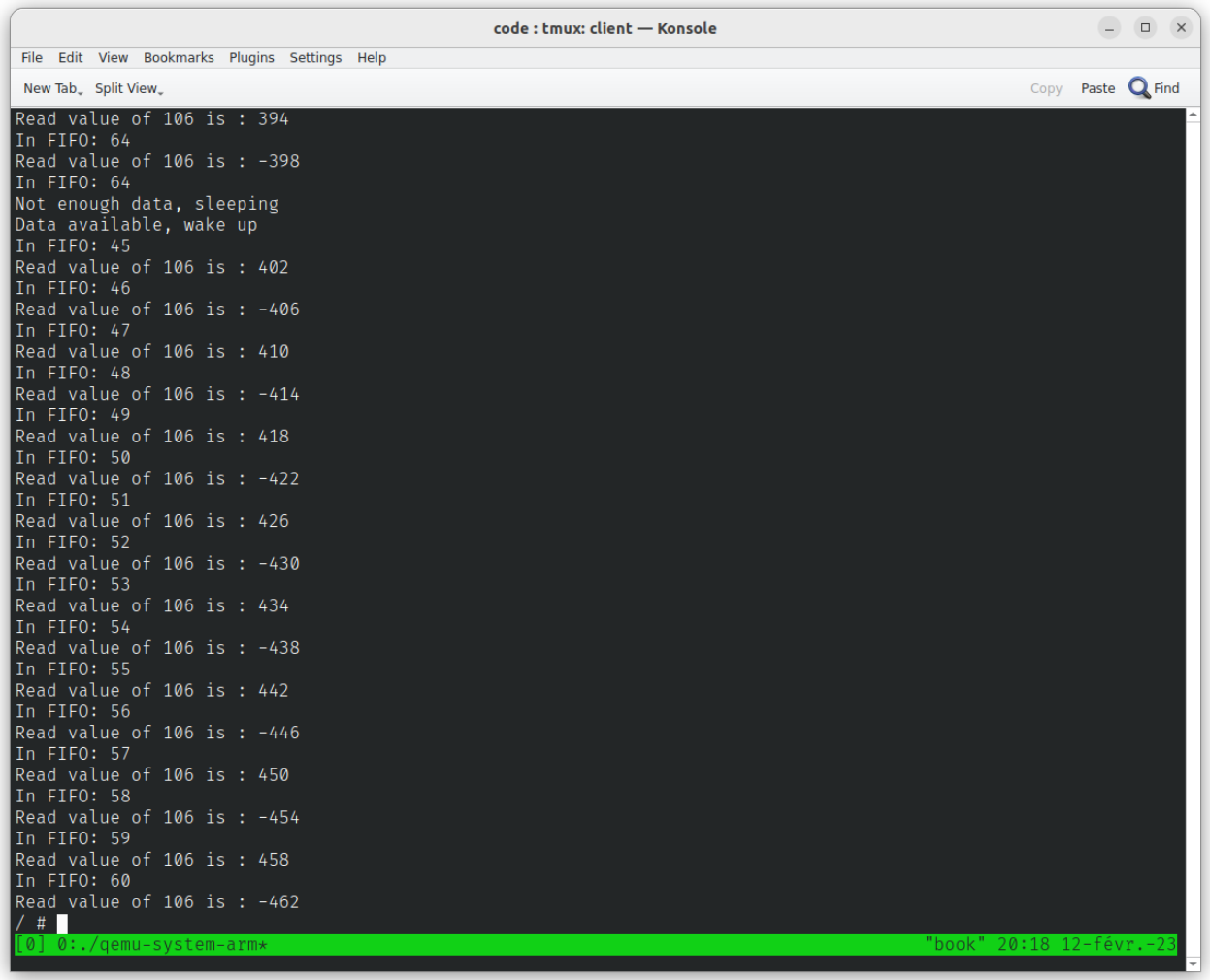


Figure 16: Output of test program

References

- Linux documentation, from `https://docs.kernel.org/`
- IOCTL in Linux, from
  `https://embetronicx.com/tutorials/linux/device-drivers/ioctl-tutorial-in-linux/`
- Process PID in kernel, from
  `https://stackoverflow.com/questions`
  `/11915728/getting-user-process-pid-when-writing-linux-kernel-module`
- Old lecture slides, from
  `https://perso.telecom-paristech.fr/duc/cours/linux/index_seti.html`